FRONTEND · BIMESTRE 02

Pré Compiladores e Typescript

Aula 05. Código type-safe reutilizável, pré compiladores como entry points, e eliminação de redundância no backend.

De onde chegamos até aqui

Frontend começou como HTML + JS soltos. Evoluiu para Vite com HMR. Agora: abandona JS puro e abraça TypeScript (.ts). Estrutura clara: `resources/` (fonte) → Vite compila → `public/` (JS puro para navegador).

O Vite passa a ser um pré compilador profissional: minifica, bundla, otimiza. Dois modos: `npm run dev` (watcher, desenvolvimento) e `npm run build` (build final, produção).

Tipos são essenciais para escalar e organizar. Generics para reutilizar sem perder type-safety.

O Navegador Só Entende JavaScript Puro

O navegador só fala uma língua: JavaScript puro. Tudo que fazemos (DOM, eventos, requisições) vem da Vendor API:

console.js

```
window.document  // Vendor API: DOM
window.fetch  // Vendor API: HTTP
window.localStorage  // Vendor API: Storage
```

TypeScript? Sass? O navegador não entende. Precisa ser transpilado para JS puro primeiro.

O Problema: Formatos que o Navegador Não Entende

Se você enviar esses formatos direto para o navegador:

erro.ts

```
// arquivo: calcular.ts
function calcular(valor: number): number { }
// ✗ SyntaxError: Unexpected token ':'
```

O navegador rejeita TypeScript. Precisa ser transpilado para JS puro antes.

TypeScript: Descobrir Erros Antes de Enviar

TypeScript adiciona tipagem. Um arquivo .ts é **transpilado** para .js puro. Erros descobertos sem rodar o código.

calcular.ts

```
function calcular(valor: number) { }
→ calcular("hello")  // ✗ Erro em transpilação
↓ (transpilado para)
// calcular.js
function calcular(valor) { }
```

Transpilação invisível: erros descobertos antes de enviar.

Generics: Uma Função para Infinitos Tipos

Sem generics, duplica função para cada tipo. Generics com `<T>` resolvem:

generics.ts

```
function wrap<T>(v: T): T { return v; }
→ wrap("hello")  // T = string
async function userListApi(): Promise<Usuario[]> {
  const r = await clientApi.get<Usuario[]>("/users");
}
```

Uma função, infinitos tipos, type-safe.

Pré Compiladores: TypeScript → JavaScript

Um pré compilador transforma código em linguagem expressiva em código que o navegador entenda:

- **TypeScript** → JavaScript
- **Sass/LESS** → CSS
- **JSX** → JavaScript

O navegador não entende TypeScript. Antes de enviar para produção (ou de servir ao desenvolvedor), precisa compilar.

Estrutura: resources/ → public/

Novo padrão: arquivos .ts, assets em `resources/` (origem). Vite compila para `public/` (JS puro).

```
resources/
├─ App.ts
├─ api.ts
└─ index.html

↓ npm run dev (watcher) ou npm run build (pontual)

public/
├─ App.a1b2c3.js (minificado)
└─ api.x9y8z7.js (minificado)
```

Minificação e Bundling

- **Minificação** — remove espaços, comentários, renomeia variáveis; 50KB → 15KB
- **Bundling** — agrupa múltiplos .js em poucos bundles; 50 requisições → 2-3

`npm run dev`: sem minificação (debug legível). `npm run build`: tudo otimizado.

Watcher: npm run dev vs npm run build

Watcher é um "observador" que fica escutando mudanças:

```
npm run dev: inicia watcher → Vite observa resources/ → Você salva App.ts → Vite recompila → HMR notifica navegador → Recarrega módulo
npm run build: compilação pontual → Lê tudo, minifica, bundla → Popula public/, termina
```

Watcher economiza tempo: não precisa build manual a cada mudança.

Vite: Duas Serventias (HMR + Pré Compilação)

- → **HMR** — alterações refletidas instantaneamente (conforto)
- → **Pré Compilação** — **a verdadeira potência**: TypeScript → JS puro automaticamente

```
App.ts → Vite transpila → App.js → navegador Invisível: salva, já vê atualizado via HMR
```

Sem Vite: build manual antes de deploy. Com Vite: invisível e confiável.

Do node_modules para o Navegador

Antes: Axios, Bootstrap, fontes → CDN da internet → carregadas direto no HTML

Agora: Tudo em node_modules/ → Vite compila → um bundle único → navegador

```
npm install axios bootstrap

// App.ts
import axios from 'axios'
import 'bootstrap/css/bootstrap.css'

↓ (Vite compila tudo junto)

navegador recebe: App.js + CSS + tudo otimizado
```

Versionamento explícito, tudo compilado, zero CDN externo.

Vite: Entry Point Containerizável

O vite.config.js já está preparado para Docker (porta 5173, separado do backend):

```
root: 'resources' // entrada aqui
server.host: true // 0.0.0.0 (Docker-ready)
server.port: 5173 // porta isolada
IS_CONTAINER !== "TRUE" → não abre browser em Docker
```

Cada entry point (Vite, backend, DB) em seu próprio container → separação, escalabilidade.

API Contextual: O Problema da Redundância

Hoje, passamos IDs na URL:

apis.js

```
DELETE /users/42  // ID na URL
// Mas o JWT já diz quem é!
PUT /users/7/tasks/100  // ID do usuário na URL
```

Redundância perigosa: o backend precisa verificar se ID na URL bate com JWT. Muitos pontos de falha.

API Contextual: ID vem do Token

Antes: DELETE /users/42 (JWT id=7). **Depois**: DELETE /users/me (uma fonte de verdade).

```
// Refatoração do projeto: userDeleteApi(): DELETE /users/me taskUpdateApi(taskId): PUT /me/tasks/{taskId}
userListApi(): GET /me
```

Menos parâmetros, menos verificações, mais seguro.

Resumo: Quatro Eixos Essenciais

- → **Vendor API + Transpilação** — navegador (e Node) só entendem JS puro
- → **TypeScript + Generics** — tipos essenciais para escalar; reutilização sem perder type-safety
- → **Vite: Pré Compilação + Watcher** — TS → JS invisível; npm run dev (continuo) vs npm run build (pontual)
- → **API Contextual** — contexto do JWT, não da URL; segurança, clareza, escalabilidade

Próximos: pasta `resources`, JS → TS, APIs para contextual. Tipos são a base para código que escala.